

Aug 27th, 5:30 PM - 5:55 PM

A Performance Comparison of Data Encryption Algorithms

Aamer Nadeem

National University of Science and Technology, Rawalpindi, Pakistan

Dr. M. Younus Javed

National University of Science and Technology, Rawalpindi, Pakistan

Follow this and additional works at: <https://ir.iba.edu.pk/iciict>



Part of the [Databases and Information Systems Commons](#), and the [Theory and Algorithms Commons](#)

Recommended Citation

Nadeem, A., & Javed, D. Y. (2005). A Performance Comparison of Data Encryption Algorithms. International Conference on Information and Communication Technologies. Retrieved from <https://ir.iba.edu.pk/iciict/2005/2005/33>

This document is brought to you by *iRepository*. For more information, please contact irepository@iba.edu.pk.

A Performance Comparison of Data Encryption Algorithms

Aamer Nadeem, Dr M. Younus Javed

Department of Computer Engineering, College of Electrical and Mechanical Engineering,
National University of Sciences and Technology,
Rawalpindi, Pakistan.

aamernadeem@yahoo.com, myjaved@ceme.edu.pk

Abstract—The principal goal guiding the design of any encryption algorithm must be security against unauthorized attacks. However, for all practical applications, performance and the cost of implementation are also important concerns. A data encryption algorithm would not be of much use if it is secure enough but slow in performance because it is a common practice to embed encryption algorithms in other applications such as e-commerce, banking, and online transaction processing applications. Embedding of encryption algorithms in other applications also precludes a hardware implementation, and is thus a major cause of degraded overall performance of the system. In this paper, the four of the popular secret key encryption algorithms, i.e., DES, 3DES, AES (Rijndael), and the Blowfish have been implemented, and their performance is compared by encrypting input files of varying contents and sizes, on different Hardware platforms. The algorithms have been implemented in a uniform language, using their standard specifications, to allow a fair comparison of execution speeds. The performance results have been summarized and a conclusion has been presented. Based on the experiments, it has been concluded that the Blowfish is the best performing algorithm among the algorithms chosen for implementation.

Key Words: Data encryption, Cryptography algorithms, Performance analysis.

I. INTRODUCTION

The main consideration in designing an encryption algorithm has to be the security of the algorithm against undesirable attacks. However, in the real world, performance and implementation cost is also important concerns. In this paper, security of the algorithms against attacks has not been compared. The primary focus is on comparing the encryption algorithms on the basis of their performance and ease of implementation.

In this paper, the security issues have been completely ignored, and only the performance of the leading secret key algorithms has been compared on different platforms, using input data files of varying sizes and formats.

While comparing the performance of algorithms, the time required to set up the key(s) has been ignored.

The rest of this paper is organized as follows: section 2 gives a brief introduction of the algorithms that have been chosen for implementation; section 3 provides implementation details; section 4 presents performance results; section 5 discusses the trade-offs between performance and security; and finally section 6 concludes the work.

II. IMPLEMENTED ALGORITHMS

The following secret key encryption algorithms were chosen for implementation, i.e.

- DES
- Triple DES
- AES (Rijndael)
- Blowfish

A. Data Encryption Standard (DES)

DES (Data Encryption Standard) is currently the most widely used block cipher in the world. In May 1973, NIST (then NBS) called for possible encryption algorithms for use in unclassified adopted encryption algorithm and is in many standards around the world (e.g. Australian Standard AS2805.5-1985). One of the largest users of the DES is the banking industry. It is for this use that the DES was primarily standardized, with ANSI reconfirming its use for 5 year periods – in future it will be replaced with AES.

Although the DES standard is public, the design criteria used are classified. There has been considerable controversy over the design, particularly in the choice of a 56-bit key [1].

B. Triple DES (3DES)

The triple DES (3DES) algorithm was needed as a replacement for DES due to advances in key searching. 3DES is a proposal based on the existing DES, and was standardized in ANSI X9.17 & ISO 8732 and in PEM for key management. It was also proposed for general EFT standard by ANSI X9 [1]. It is backwards compatible with existing single DES (when $K_1=K_2=K_3$). The 3DES algorithm uses either two or three 56-bit keys. Thus the effective key length is up to 168 bits. 3DES is defined by the following function:

$$C = \text{DES}_{K_3} \{ \text{DES}_{-1} \text{DES}_{K_1}(P) \}$$

where P = Plaintext

C = Ciphertext

DES_K = DES encryption using key K

DES_{-1}

DES_K = DES decryption using key K

C. Advanced Encryption Standard (AES)

In September 1997, US NIST announced a call for candidate ciphers for its new Advanced Encryption Standard (AES), because clearly a replacement for DES was needed at that time [2]. The candidate ciphers were to be submitted by June 1998, and a finalist was selected in October 2000. In total 15 candidates were accepted in June 98 (6 were rejected as incomplete), and 5 were short-listed in August 99. Finally, Rijndael was selected as the AES finalist in October 2000.

NIST has released all submissions and unclassified analyses. The AES candidates are the latest generation of block ciphers, and have a significant increase in the block size - from the old standard of 64-bits up to 128-bits; and keys from 128 to 256-bits. In part this has been driven by the public demonstrations of exhaustive key searches of DES & RC-5 (at 64-bits).

After exhaustive analysis and evaluation rounds, Rijndael, designed by Rijmen & Daemen in Belgium [4], was selected as AES finalist [3]. It has the following attributes:

1. 128-bit block size
2. 128, 192, or 256 bit key size
3. an iterative rather than a Feistel cipher (like IDEA)
4. treats data as 4 groups of 4 bytes
5. has 9, 11, or 13 rounds, where each round consists of:
 - a byte substitution step (1 S-box used)
 - on every byte)
 - a shift rows step (shuffle the bytes)
 - between groups)
 - a mix columns step (matrix multiplication of groups with each other)
 - an add round key step
6. all operations can be combined into xor and table lookups – hence implementation can be very fast and efficient.

D. Blowfish

Most of the encryption algorithms today are unavailable to the public – many of them are protected by patents (e.g. Khufu, REDOC II, and IDEA), or being kept secret by the governments (e.g. Skipjack and Capstone are protected by the U.S. government). Many of the other algorithms are available only in part (e.g. RC2, RC4, and GOST). *Bruce Schneier* – one of the world's leading cryptologists, and the president of *Counterpane Systems*, a consulting firm specializing in cryptography and computer security – designed the Blowfish algorithm [5] and made it available in the public domain. Blowfish is a variable length key, 64-bit block cipher. It was his intent from the outset of creating this new encryption algorithm to provide the world with a new encryption standard. The Blowfish algorithm was first introduced in 1993, and has not been cracked yet. It is also noteworthy to point out that this algorithm can be optimized in hardware applications, although it, like most other ciphers, is often used in software applications.

III. IMPLEMENTATION DETAILS

The Java platform (JDK 1.4) was used to implement the above algorithms. The following

are some of the main reasons explaining why Java was chosen:

- Java is considered platform independent because Java compiler produces byte code rather than machine code for a specific type of hardware – this feature of Java makes sure that the programs will run on any platform (with Java interpreter). Thus, the implemented algorithms can be tested on a variety of platforms for comparison purposes.
- Java (in particular JDK 1.4) provides a large library of built-in classes and methods (in the form of API) that assist the programmer in writing code for cryptographic algorithms. For example, the BigInteger class in Java lets the programmer apply arithmetic and bit manipulation operations on integer values of arbitrarily large sizes.
- Conversion from integer to string and vice versa, and likewise conversion of integer values from one radix to another is relatively easier in Java due to the built-in routines provided for this purpose. For instance, one frequently needs to convert a decimal value to binary or hexadecimal, and vice versa.
- The concepts of object serialization and stream input/output make it easy to read and write objects to external disk files

However, using Java to implement cryptography algorithms has some drawbacks as well. The main drawback of using Java is its slow speed – this is because Java compiler does not generate native machine code, rather it produces an intermediate form code (called byte code) which needs an interpreter to run. This could have been a concern because the performance of various algorithms had to be tested, but the effect of inefficiency was balanced out because all the algorithms were implemented in the same language (Java) and were tested on the same platform.

As mentioned earlier, the primary goal of this research was not to have the most efficient

implementation of cryptography algorithms – but just to compare the relative performance of various popular algorithms. So the algorithms were implemented as is, using a uniform language, and were tested on a uniform platform. Each of the above algorithms was implemented as a Java class. In the subsequent sections, for each algorithm, a description of the instance variables and the implemented methods is provided.

IV. PERFORMANCE RESULTS

All the implementations provided and discussed here have been coded from scratch using the specification documents of respective algorithms.

A. Performance Measurement Approach

A fairly consistent style across the source code was adopted to ensure a fair comparison. All the routines have been implemented as per specifications without any bias towards encryption and decryption speed, or to key set-up or to memory use in tables. The main reason for not optimizing the code for performance was to measure only *relative* performance of algorithms. Moreover, techniques for enhancing performance of algorithms generally also make the source code more obscure and unreadable.

All the algorithms and results reported are based on implementation using Java language (JDK 1.4). In general, custom routines were written for helper methods such as left-shift operation, binary to hex conversion and vice versa, xor operation on two bit strings etc. instead of using library functions. However, whenever convenient, Java's built-in class library was also used, for instance the BigInteger class was used to manipulate integer values of arbitrarily large size such as keys and data blocks.

B. Measuring Execution Times

Obtaining accurate and repeatable execution time measurements proved to be more difficult than was originally anticipated. After many experiments, it was decided to settle on the use of a Pentium-II 266 MHz machine (running Microsoft Windows operating system) and a Pentium-4, 2.4 GHz machine (running Microsoft Windows XP operating system) as the basis for time measurements. Because the primary goal

was to measure the encryption times of algorithms, the initialization and key set-up times were excluded from the comparison. Also, because decryption time is generally the same as encryption time for almost all the algorithms, therefore only the encryption times were measured.

C. Performance Results for Block Ciphers

The execution results of secret key algorithms in ECB (Electronic Codebook) mode are presented first. These results are shown Table 1 and Table 2, for execution of code on a Pentium-II 266 MHz machine, and Pentium-4, 2.4 GHz machine respectively.

Table 1: Comparative execution times (in seconds) of secret key algorithms in ECB mode on a Pentium-II 266 MHz machine

<u>Input size (bytes)</u>	<u>DES</u>	<u>3DES</u>	<u>AES</u>	<u>BF</u>
20,527	24	72	39	19
36,002	48	123	74	35
45,911	57	158	94	46
59,862	74	202	126	58
69,646	83	243	143	67
137,325	160	461	285	136
158,959	190	543	324	158
166,364	198	569	355	162
191,383	227	655	378	176
232,398	276	799	460	219
Average time	134	383	228	108
Bytes/sec	835	202	491	1,036

Table 2: Comparative execution times (in seconds) of secret key algorithms in ECB mode on a Pentium-4, 2.4 GHz machine

<u>Input size (bytes)</u>	<u>DES</u>	<u>3DES</u>	<u>AES</u>	<u>BF</u>
20,527	2	7	4	2
36,002	4	13	6	3
45,911	5	17	8	4
59,862	7	23	11	6
69,646	9	26	13	7
137,325	17	51	26	14
158,959	20	60	30	16
166,364	21	62	31	17
191,383	24	72	36	19
232,398	30	87	44	24
Average time	14	42	21	11
Bytes/sec	7,988	2,663	5,326	10,167

An obvious way to compare the performance of these algorithms is to take average of execution times for each algorithm, and rank them accordingly. Using this criterion, it is apparent from Tables 1 and 2 that the four symmetric

key algorithms are in the following order, as regards their performance:

1. Blowfish (fastest)
2. DES
3. AES
4. Triple DES (slowest)

In general, the performance of a block cipher varies with the block size and the key size. The larger the block size, the faster will be the algorithm, because with a larger block size, a large chunk of the data will be encrypted in a single execution cycle of the algorithm. With a smaller block size, the same size of input data would require more execution cycles of the algorithm, thus increasing the overall encryption time other things being equal. Likewise, a larger key results in a slower algorithm, because in general, all bits of the key are involved in an execution cycle of the algorithm. With a smaller key, a lower number of key bits are involved, thus reducing the time to complete one execution cycle.

D. Performance Results for Stream Ciphers

In this section, the results of execution of the secret key algorithms in CFB mode are presented. Tables 3 and 4 give execution times of these algorithms on Pentium-II 266 MHz, and Pentium-4, 2.4 GHz platforms respectively.

Table 3: Comparative execution times of secret key algorithms in CFB mode on a Pentium-II 266 MHz machine

<u>Input size (bytes)</u>	<u>DES</u>	<u>3DES</u>	<u>AES</u>	<u>BF</u>
20,527	188	498	598	141
36,002	362	934	1123	271
45,911	431	1174	1484	352
59,862	570	1532	1932	433
69,646	629	1894	2251	518
137,325	1203	3622	4419	992
158,959	1405	4240	5044	1175
166,364	1511	4424	5501	1216
191,383	1714	5128	5923	1369
232,398	2139	6238	7231	1649
Average time	1,015	2,909	3,551	812
Bytes/sec	110	38	31	138

As with block ciphers, the performance of a stream cipher also varies with the block size and the key size, but the effect of a larger block size is reversed. The larger the block size, the slower will be the algorithm, because with a larger block size, the algorithm will have to do more work for

the same amount of input data (a bit or a byte) in a single execution cycle of the algorithm. With a smaller block size, the same size of input data would be encrypted more efficiently, thus decreasing the overall encryption time other things being equal.

Table 4: Comparative execution times of secret key algorithms in CFB mode on a Pentium-4, 2.4 GHz machine

<u>Input size</u> <u>(bytes)</u>	<u>DES</u>	<u>3DES</u>	<u>AES</u>	<u>BF</u>
20,527	17	58	62	18
36,002	30	99	94	23
45,911	41	130	125	33
59,862	52	181	174	46
69,646	69	201	200	53
137,325	129	401	409	104
158,959	151	472	473	122
166,364	159	488	489	130
191,383	185	568	567	148
232,398	229	681	687	184
Average time	106	328	328	86
Bytes/sec	1055	341	341	1300

The effect of a larger key in a stream cipher is the same as in a block cipher, i.e. it slows down the encryption, because in general, all bits of the key are involved in an execution cycle of the algorithm. With a smaller key, a lower number of key bits are involved, thus reducing the time to complete one execution cycle.

V. PERFORMANCE VS. SECURITY TRADE-OFFS

The work presented here is primarily about performance but it does not seem sensible to consider this in complete isolation from security since there are inevitably trade-offs between these attributes in the real world scenario. One obvious example is the number of rounds employed by each algorithm – a large number of rounds make the algorithm slower but are supposed to provide greater security [6].

Likewise, the quality of each round in achieving the *mixing* that the cipher is intended to provide is also a factor that affects performance of an algorithm. Thus, there is always a trade-off between security and performance in similar algorithms – such as symmetric key algorithms. Eli Biham [7] has suggested that performance of algorithms should be measured by timing the *minimum number of secure rounds* for each algorithm – that is the estimated number of

rounds needed to make a brute force key search the most efficient form of attack. This is a controversial suggestion that some cryptologists do not accept even in principle. Therefore, it is proposed to look at this as follows:

- the principle of measuring performance of an algorithm by timing the *minimum number of secure rounds* for each algorithm is useful, but
- there is no easy way of obtaining impartial and widely accepted values for the minimum number of secure rounds for each algorithm.

However as an AES algorithm contributor, Eli Biham is not seen as an impartial observer and it would hence be preferable to have a wider community basis for setting the values to be used in such a process.

A. Implementation and Security Issues

There are a significant number of algorithms that are considered to be secure at an algorithm level. In practice, however, most failures in cryptographic systems derive not from weaknesses in the algorithms used but rather from the exploitation of subtle flaws in the way the algorithms are implemented or through the exploitation of interactions between algorithm implementations and the environments in which they operate. This means that it is vital to judge implementations not simply from the perspective of the performance that they provide but also from the perspective of *implementation assurance* i.e. can one be confident that the implementation does what the algorithm requires, no more and no less, and can one also be confident that it does not interact with the wider environment in ways that can be exploited to undermine the security that the algorithm itself is otherwise capable of providing. To answer these questions much more hard evidence needs to be derived from the implementation of the algorithms in environments that are representative of practical use. In particular, there is very little evidence at the moment of how the algorithms behave in low-end environments of the kind that may apply in much of electronic commerce and consumer applications.

There is also a need to understand whether the different algorithms are architecturally different in their ability to support implementation

assurance techniques. None of the existing specifications are precise enough to support formal design and analysis methods and it even seems possible that at least some of the algorithms on offer are sufficiently complex to make these techniques difficult or even impossible to apply. This may also be true of other approaches for implementation assurance. This is an area where much more work is needed before any final algorithm choices are made.

VI. CONCLUSION AND FUTURE WORK

In this paper, the popular secret key algorithms including DES, 3DES, AES (Rijndael), Blowfish, were implemented, and their performance was compared by encrypting input files of varying contents and sizes. The algorithms were implemented in a uniform language, using their standard specifications, and were tested on two different hardware platforms, to compare their performance. In the end, the results were presented which conclude that the Blowfish is the fastest algorithm. Though security was not catered for, in practice, however, one would consider the security first. A proposed direction for the future work could be to analyze the performance/security trade-off in greater depth. For instance, an algorithm with more complex rounds and a larger number of rounds is generally considered more secure. The impact of these and other such factors on the overall performance of an algorithm needs to be measured.

REFERENCES

- [1] National Bureau of Standards – Data Encryption Standard, FIPS Publication 46, 1977.
- [2] NIST, "Advanced Encryption Standard Call", NIST, 1997. <http://www.nist.gov/aes/>
- [3] NIST Advanced Encryption Standard (AES) Development Effort web site <http://csrc.nist.gov/encryption/aes/aes-home.htm>
- [4] Daemen, J., Rijmen, V.: "AES Proposal: Rijndael", Banksys/Katholieke Universiteit Leuven, Belgium, AES submission, Jun 1998.
- [5] Schneier, B.: "Description of a New Variable-Length Key, 64-Bit Block Cipher (Blowfish)", Fast Software Encryption, Cambridge Security Workshop Proceedings (Dec. 1993), Lecture Notes in Computer Science (LNCS), Springer-Verlag, Vol. 809, pp. 191-204, 1993, ISBN 3-540-58108-1.
- [6] Schneier, B., Kelsey, J., Whiting, D., Wagner, D., Hall, C., and Ferguson, N.: "Performance Comparison of the AES Submissions", Counterpane Systems, Dec 1998. <http://www.counterpane.com/AESperformance.html>
- [7] Biham, E.: "Design Tradeoffs of the AES Candidates", in *Asiacrypt'98*, Lecture Notes in Computer Science, Springer-Verlag, 1998.